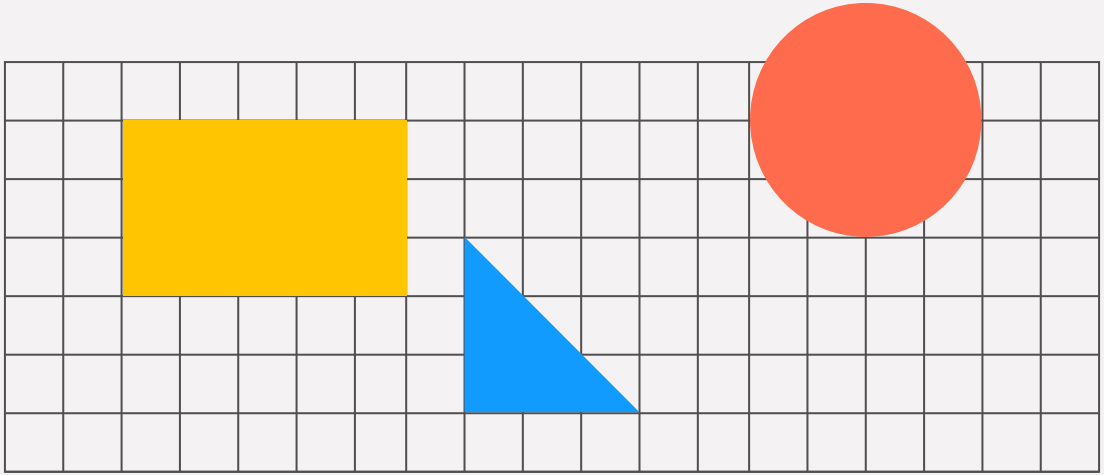
# Database Systems Presentation

Group 2 Members:

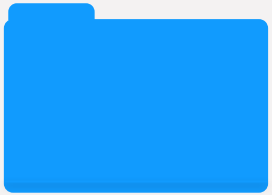
| | |
|---|---|
| George Tang Yunn Shun | 0139228 |
| Jordan Lim Min Yong | M44100033 |
| Edison Tan Zy Young | 0139324 |
| Muhammad Zahran | 0139186 |
| Gee Lok Thim | 0139385 |

Course : Database Systems XDCS1054N
Lecturer : Ms Norsyela

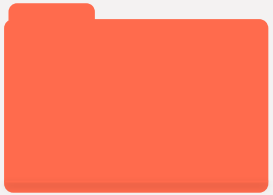




Introduction



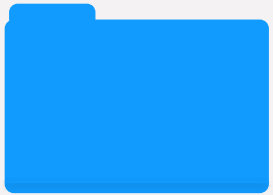
System Design



Database Implementation



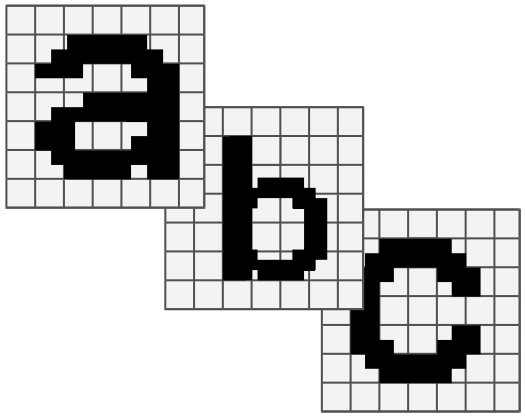
Real Life Examples



Suggestions for Improvement

# Introduction

- *Summarizes key components of Task 1*
- *Explains real-life use of database systems*
- *Covers industry, ERD, and implementation*
- *Includes real-world examples*
- *Identifies system limitations*
- *Suggests practical improvements*
- *Aligns with project goals*



# Identify DBS System

A Library Management System is a system to manage the library and track books available by looking at the ISBN, Author Name, Title, etc. This also includes a system to track members if they are loaning any books, the loan date and due date and or any penalties charged.

# Key Feature & Component

**Key Features:**
LMS helps libraries to run better such as ensuring data accuracy, if the book's information is wrong, instead of finding it manually we can search it by using CRUD operations to modify the information.

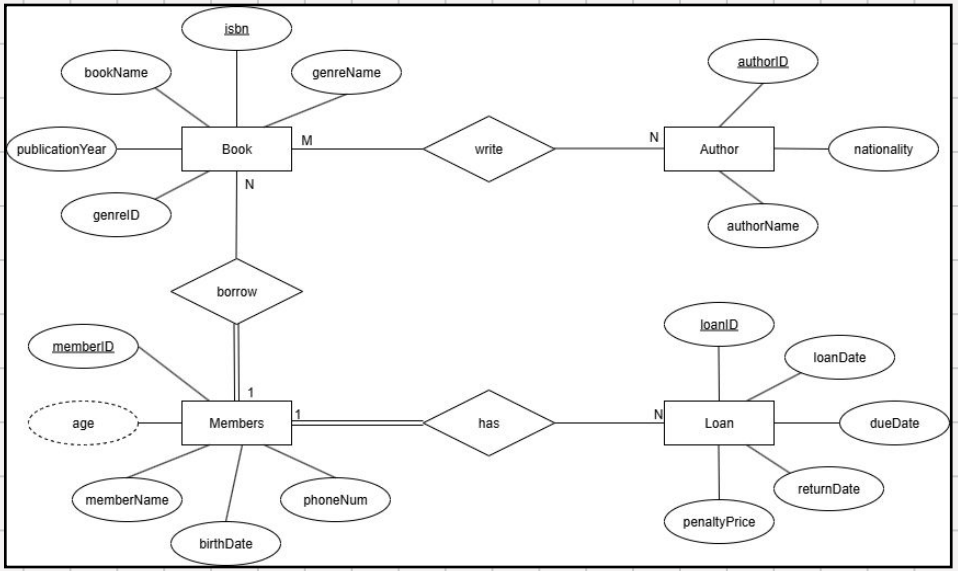
**Components:**
Book:
Attributes: isbn (PK), bookName, publicationYear, genreName, genreID
Purpose: Store the book's information.

Members:
Attributes: memberID (PK), age (derived attribute), memberName, phoneNum, birthDate
Purpose: Store the member's information

Author:
Attributes: authorID (PK), authorName, nationality
Purpose: Store the author's details

Loan:
Attributes: loanID (PK), loanDate, dueDate, returnDate, penaltyPrice
Purpose: Keep track of the member's loaning information

# Entity Relationship Diagram (ERD)




**The following are the relationships between the entities:**

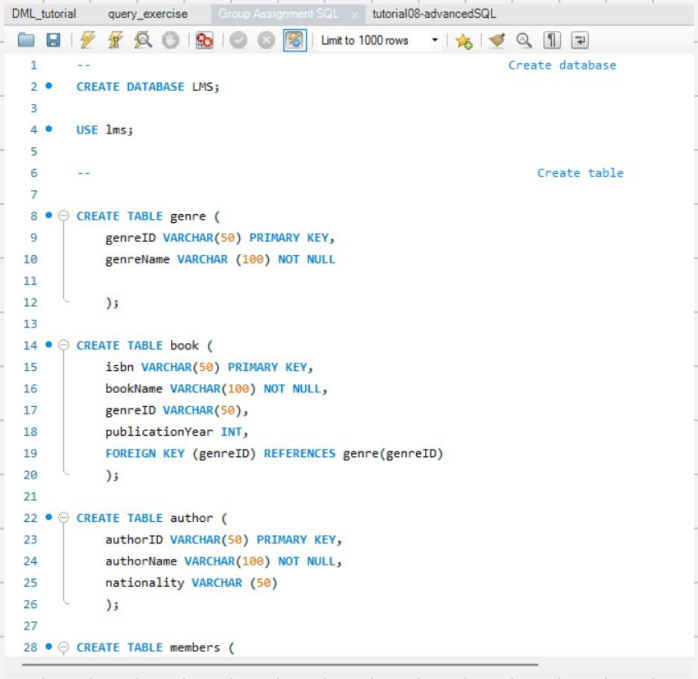
- An author can write many books and a book can belong to many authors.
- A member can borrow many books, but each book must only belong to one member.
- A member can have many loans, but a loan must only belong to one member.

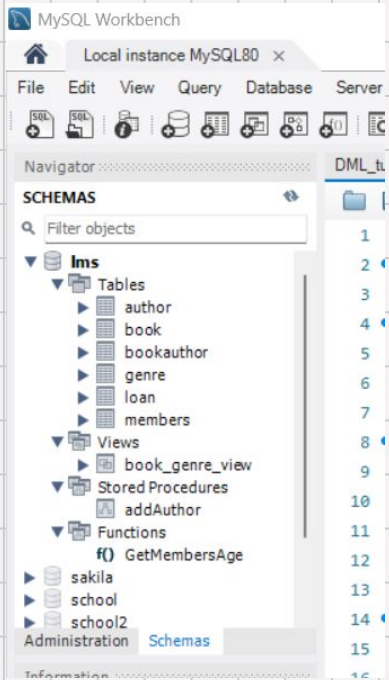
**A brief explanation:**

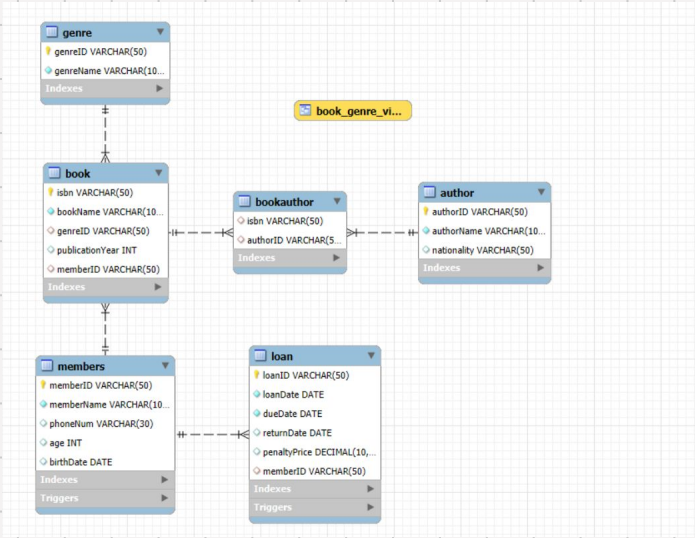
The Members entity is for the visitor who wants to apply for a membership so they can loan a book. The loan entity is to track which book and when the book is loaned and whether there is any penalty given. The Author entity is information about the respective authors and lastly the Book entity is information of the books.

# Database Creation

**Created using MySQL Workbench 8.0 CE**



# Database Creation

**Tables involved in the Database**

| Book | |
|---|---|
| PK | isbn |
| | bookName |
| | publicationYear |
| FK | genreID |
| FK | memberID |

| Genre | |
|---|---|
| PK | genreID |
| | genreName |

| Members | |
|---|---|
| PK | memberID |
| | memberName |
| | phoneNum |
| | birthDate |
| | age |

| Loan | |
|---|---|
| PK | loanID |
| | loanDate |
| | dueDate |
| | returnDate |
| | penaltyPrice |
| FK | memberID |

| Author | |
|---|---|
| PK | authorID |
| | authorName |
| | nationality |

| BookAuthor | |
|---|---|
| PK, FK | isbn |
| PK, FK | authorID |

**Book: <PK>isbn, bookName, publicationYear, <FK>genreID, <FK>memberID**

**Genre: <PK>genreID, genreName,**

**Members: <PK>memberID, memberName, phoneNum, birthDate, age (derived attribute)**

**Loan: <PK>loanID, loanDate, dueDate, returnDate, penaltyPrice, <FK>memberID**

**Author: <PK>authorID, authorName, nationality**

**BookAuthor: <PK><FK>isbn, <PK><FK>authorID**

# CRUD Operations

## CREATE

```
-- inserting data into tables (CREATE)
INSERT INTO members (memberID, memberName, phoneNum, age)
VALUES
('CR001', 'Bobby Meng Leng', '019-2321123', 20),
('CR002', 'Jennifer Lee', '016-9001033', 18);

INSERT INTO author (authorID, authorName, nationality)
VALUES
('RS0001', 'J.K. Rowling', 'United Kingdom'),
('RS0002', 'Stephen King', 'United States'),
('RS0003', 'Victor Axelson', 'Denmark');
```

**Inserting values into the table**

## READ

```
-- Viewing the data (READ)
SELECT * FROM members
WHERE age = 20;

SELECT * FROM author;
```

**Viewing the tables/records created**

# CRUD Operations

## UPDATE

```
-- Updating the data (UPDATE)
UPDATE members
SET age = 22
WHERE memberID = 'CR002';

UPDATE author
SET nationality = 'France', authorName = 'Jean Paul Gaultier'
WHERE authorID = 'RS0001';
```

**Updating records that currently exist in the table**

## DELETE

```
-- Deleting data (DELETE)
DELETE FROM author
WHERE authorID = 'RS0003';

DELETE FROM members
WHERE memberID = 'CR002' ;
```

**Deleting existing records**

# Advanced SQL Objects

## View

```sql
-- Views (Jordan)
CREATE VIEW book_genre_view AS
SELECT book.isbn, book.bookName, genre.genreName
FROM book
JOIN genre ON book.genreID = genre.genreID;

SELECT * FROM book_genre_view;
```

**Hide unnecessary columns and provide easier access to related data.**

# Advanced SQL Objects

## Stored Procedure

```sql
-- Stored procedure (Lok Thim)

DELIMITER \\

CREATE PROCEDURE addAuthor (
    IN p_authorID VARCHAR(50),
    IN p_authorName VARCHAR(100),
    IN p_nationality VARCHAR (50)
)
BEGIN
    INSERT INTO author (authorID, authorName, nationality)
    VALUES (p_authorID, p_authorName, p_nationality);
END\\

DELIMITER ;

CALL addAuthor ('RS004', 'Stan Lee', 'United States');
```

**This stored procedure is created to enable easier and faster data entry for the author table**

# Advanced SQL Objects

## Function

```
-- Function (Zahran)
DELIMITER \\
CREATE FUNCTION GetMembersAge(birthDate DATE)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN TIMESTAMPDIFF(YEAR, birthDate, CURDATE());
END\\

DELIMITER ;
```

**A function to calculate the derived attribute (age) from birthDate.**

# Advanced SQL Objects

## Trigger

```
-- Trigger (Edison)

DELIMITER \\
CREATE TRIGGER after_member_insert
AFTER INSERT ON members
FOR EACH ROW
BEGIN
  INSERT INTO member_log (memberID, memberName, action_type)
  VALUES (NEW.memberID, NEW.memberName, 'INSERT');
END\\

DELIMITER ;
```

**The trigger automatically tracks new member additions by recording their ID and name into a member_log table right after they're inserted into the members table.**

# Advanced SQL Objects

## Trigger

```
-- Trigger (George)

DELIMITER \\
CREATE TRIGGER set_penalty_before_update
BEFORE UPDATE ON loan
FOR EACH ROW
BEGIN
    IF NEW.returnDate > NEW.dueDate THEN
        SET NEW.penaltyPrice = DATEDIFF(NEW.returnDate, NEW.dueDate) * 1.50;
    END IF;
END\\
DELIMITER ;
```

**This trigger is activated before a loan record is updated. If the return date is later than the due date, it calculates the number of late days and multiplies it by RM1.50 to set the “penaltyPrice”.**

# Real Life Examples:

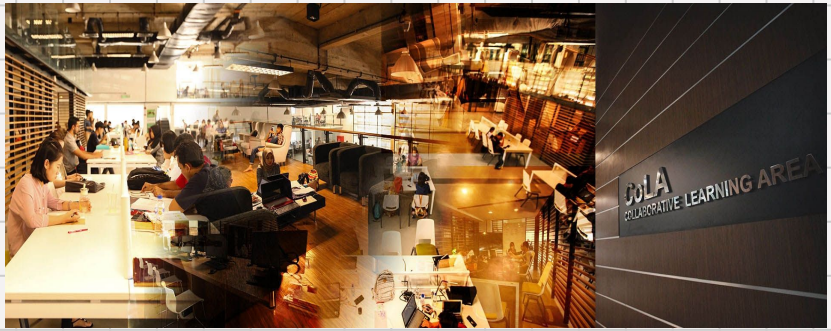
## Library Management System

- Automates all the operations within a library.
- Is used widely across corporations, research institutes, hospitals and public institutions to manage both digital and physical resources.
- A vital tool for modern libraries, enabling them to operate more efficiently, provide better service to their users, and manage their resources effectively.

## Universiti Malaya Library

Name Department Course Number Section Time Location DATE

# Universiti Malaya Library




- Top Academic library in the Malaysia, boasting its large collection of books, journals, conference proceedings, audio video, microform, theses and dissertations.
- Uses an integrated LMS system called Virtua ILS, which is built on a relational DBMS from a multi-million corporation.
- This system ensures data consistency, fast query response time, and role-based access control for students, librarians, and administrators.

It's known for its comprehensive features and scalability, particularly favored by academic libraries.

Virtua ILS

# Current Limitations

1

## Scalability Issues

Relational database management systems may encounter difficulties managing increasing data volumes as the number of digital resources and users rises

2

## Limited Ability to Integrate

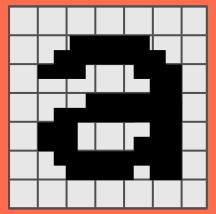
Accessibility and user convenience may be diminished by the system's difficulty connecting to contemporary resource like external journal databases, mobile applications, and e-learning platforms.

3

## High Maintenance Costs

The library management system will be more expensive to operate and less flexible in the event of future changes because the vendor is in charge of system updates, maintenance, and customizations.

# Proposed Enhancements/Innovations.



## Migration to Cloud database

Moving the library's database over to a cloud-based system like Google Cloud would make it easier for the staff to manage, decrease maintenance costs, scalability and improve its performance.



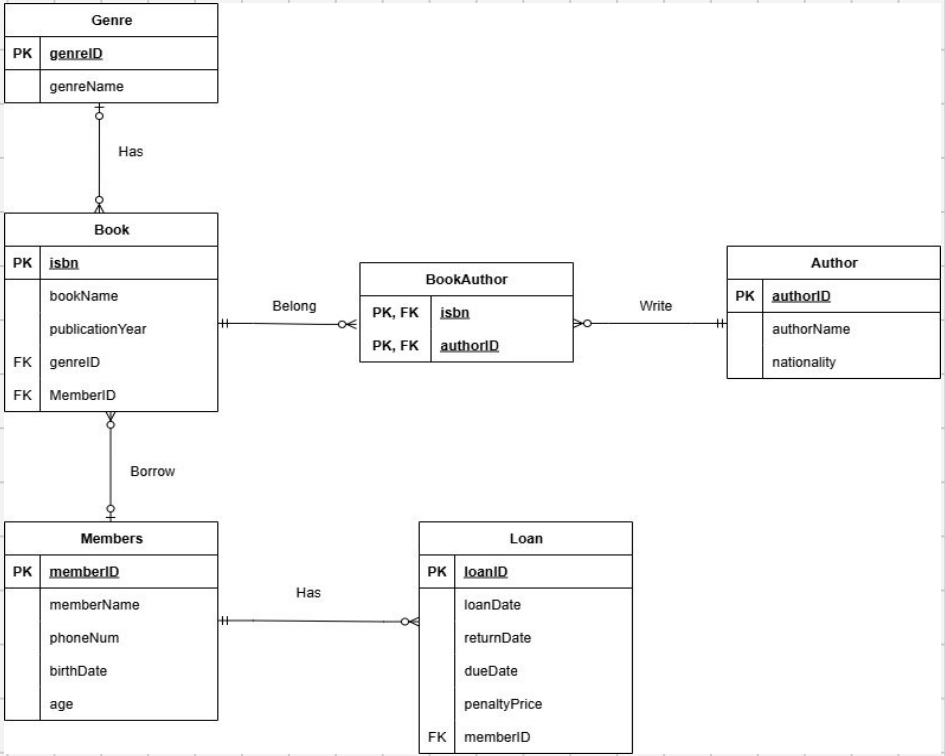
## Implementation of APIs

Creating APIs (Application Programming Interface) would make it possible to integrate other platforms like mobile apps, learning management systems, and third-party digital libraries more easily.

# Suggestion of Improvements for Current LMS

1. Judging the current crow's foot diagram, there are a few improvements able to be added to our system to ensure a more detailed and smooth process for users.

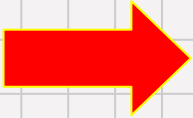
2. It is always advisable to find flaws from our work to understand our weak points and overcome the issues.

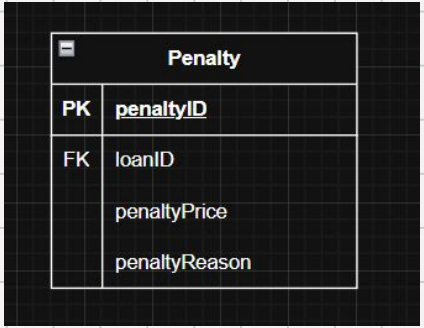


# The Improvements

1. Adding a Penalty table

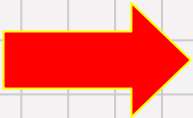
   When we add a new Penalty table, we can also include more details for the penalties, such as the amount and reason of the penalty.

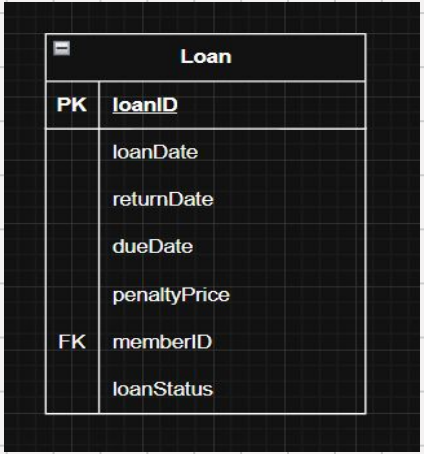


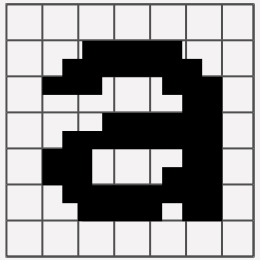


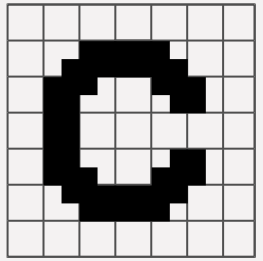

2. Adding a status attribute for the Loan table

   The addition should give the status of the book such as loaned, returned and overdue. This can help users to know more about the book.









# Thank You

